임베디드 시스템

Embedded Systems

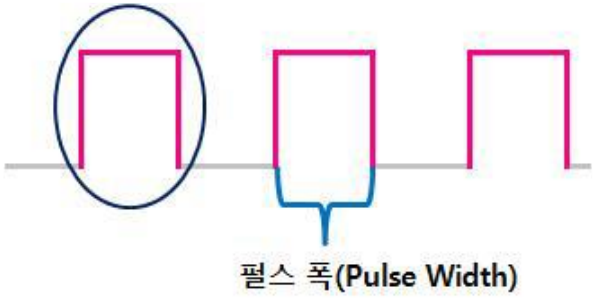
PWM | Servo Motor | Passive Buzzer | Camera

강원대학교 컴퓨터공학과 권용훈

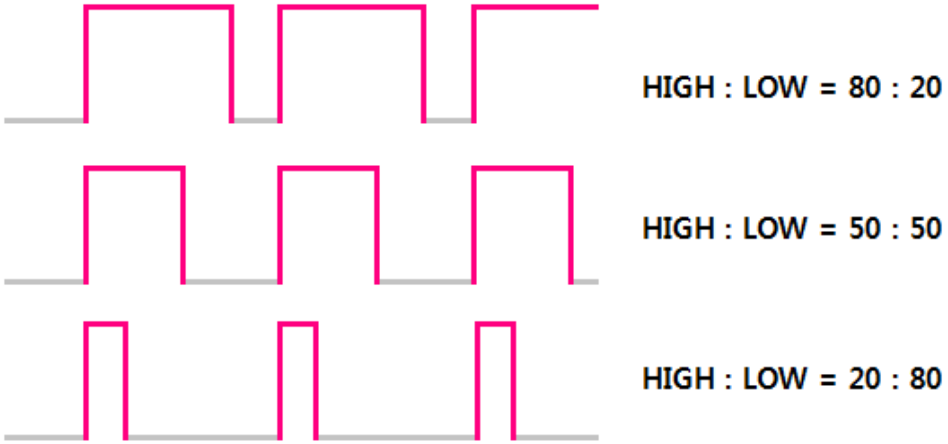
PWM(1/5)

PWM(Pulse Width Modulation)

- 펄스 폭 변조
- GPIO 핀을 통해서 HIGH와 LOW 신호 출력 비율을 조절하여 다양한 파형을 만들어 내는 것



펄스(Pulse)



PWM에 따른 파형 형태

PWM(2/5)

WiringPi의 PWM 핀

- 1, 23, 24, 26 핀

```
pi@raspberrypi:~/wiringPi $ gpio readall
 +-----+-----+---------+------+---+---Pi 3B--+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 |     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
 |   2 |   8 |   SDA.1 |   IN | 1 |  3 || 4  |   |      | 5v      |     |     |
 |   3 |   9 |   SCL.1 |   IN | 1 |  5 || 6  |   |      | 0v      |     |     |
 |   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 0 | IN   | TxD     | 15  | 14  |
 |     |     |      0v |      |   |  9 || 10 | 1 | IN   | RxD     | 16  | 15  |
 |  17 |   0 | GPIO. 0 |   IN | 0 | 11 || 12 | 0 | IN   | GPIO. 1 | 1   | 18  |
 |  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
 |  22 |   3 | GPIO. 3 |   IN | 0 | 15 || 16 | 0 | IN   | GPIO. 4 | 4   | 23  |
 |     |     |    3.3v |      |   | 17 || 18 | 0 | IN   | GPIO. 5 | 5   | 24  |
 |  10 |  12 |    MOSI |   IN | 0 | 19 || 20 |   |      | 0v      |     |     |
 |   9 |  13 |    MISO |   IN | 0 | 21 || 22 | 0 | IN   | GPIO. 6 | 6   | 25  |
 |  11 |  14 |    SCLK |   IN | 0 | 23 || 24 | 1 | IN   | CE0     | 10  | 8   |
 |     |     |      0v |      |   | 25 || 26 | 1 | IN   | CE1     | 11  | 7   |
 |   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
 |   5 |  21 | GPIO.21 |   IN | 1 | 29 || 30 |   |      | 0v      |     |     |
 |   6 |  22 | GPIO.22 |   IN | 1 | 31 || 32 | 0 | IN   | GPIO.26 | 26  | 12  |
 |  13 |  23 | GPIO.23 |   IN | 0 | 33 || 34 |   |      | 0v      |     |     |
 |  19 |  24 | GPIO.24 |   IN | 0 | 35 || 36 | 0 | IN   | GPIO.27 | 27  | 16  |
 |  26 |  25 | GPIO.25 |   IN | 0 | 37 || 38 | 0 | IN   | GPIO.28 | 28  | 20  |
 |     |     |      0v |      |   | 39 || 40 | 0 | IN   | GPIO.29 | 29  | 21  |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+---Pi 3B--+---+------+---------+-----+-----+
```

```
// Pin modes

#define INPUT            0
#define OUTPUT           1
#define PWM_OUTPUT       2
#define GPIO_CLOCK       3
#define SOFT_PWM_OUTPUT  4
#define SOFT_TONE_OUTPUT 5
#define PWM_TONE_OUTPUT  6

#define LOW              0
#define HIGH             1

// Pull up/down/none

#define PUD_OFF          0
#define PUD_DOWN         1
#define PUD_UP           2

// PWM

#define PWM_MODE_MS      0
#define PWM_MODE_BAL     1
```

PWM(3/5)

WiringPi & softPWM

- wiringPi 제공함수

void pwmWrite(int pin, int mode)		
위 치	wiringPi / wiringPi.h	
기 능	pwm 전용 핀에 pwm파형 출력	
매개변수	int pin	pwm 전용 핀, 1번, 24번 핀 중 하나
	int value	0 ~ 1024 범위 값 (0 = LOW / 1024 = HIGH)
리턴 값	void	없음

softPWM 함수

int softPwmCreate(int pin, int value, int range)		
위 치	wiringPi / wiringPi / softPwm.h	
기 능	pwm 전용 핀에 pwm파형 출력	
매개변수	int pin	pwm을 출력할 일반 GPIO 핀 번호
	int value	초기 값
	int range	pwm 범위
리턴 값	int	0 - 성공

void softPwmWrite(int pin, int value)		
위 치	wiringPi / wiringPi / softPwm.h	
기 능	software pwm 핀에 PWM 출력	
매개변수	int pin	software pwm 핀 번호
	int value	0 ~ 100 범위 값 (0 = LOW / 100 = HIGH)
리턴 값	void	없음

void softPwmStop(int pin)		
위 치	wiringPi / wiringPi / softPwm.h	
기 능	일반 핀에 pwm파형 출력을 정지	
매개변수	int pin	pwm 출력을 하고 있는 핀
리턴 값	void	없음

PWM(4/5)

LED 제어

```
#include <stdio.h>
#include <wiringPi.h>
#include <softPwm.h>
#define R_LED_PIN     27    //- PWM기능 핀


int main(void)
{
      //- wiringPi 초기화
  if( wiringPiSetup() == -1){
    printf("Setup Fail ~!!");
    return -1;
  }
  //- 소프트 PWM으로 센서 초기화
  pinMode(R_LED_PIN, SOFT_PWM_OUTPUT);
  //- PWM 출력 주파서 설정
  softPwmCreate(R_LED_PIN,0, 255);
  //- RGB LED 밝기 설정
  while(1){
    for(int idx=0; idx<=255; idx +=5){
      softPwmWrite(R_LED_PIN, idx) ;
      printf("Red LED ON ~%d\n", idx);
      delay(50);
    }
  }
  return 0;
}
```

PWM(5/5)

DC모터 제어

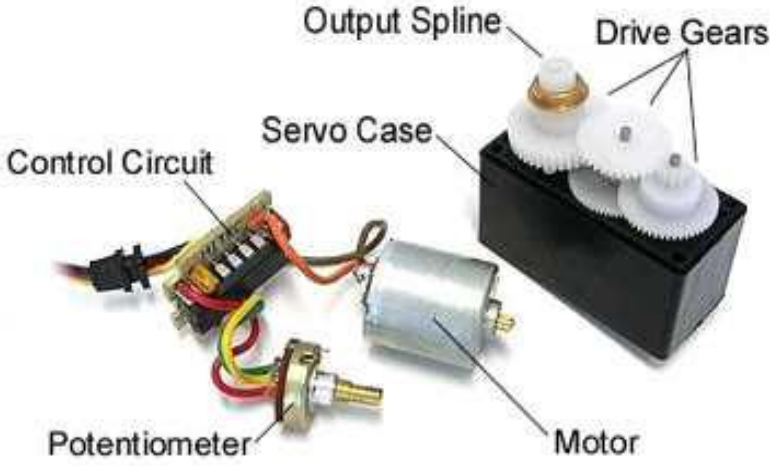
```c
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#define DCMotorA     6
#define DCMotorB     31

int main(void){
    wiringPiSetup();

    pinMode(DCMotorA, SOFT_PWM_OUTPUT);
    pinMode(DCMotorB, SOFT_PWM_OUTPUT);

    softPwmCreate(DCMotorA, 0, 100);
    softPwmCreate(DCMotorB, 0, 100);

    int duty = 10;     //- HIGH & LOW 출력 비율
    int updown = 10;      //- 듀티비 증감 값
```

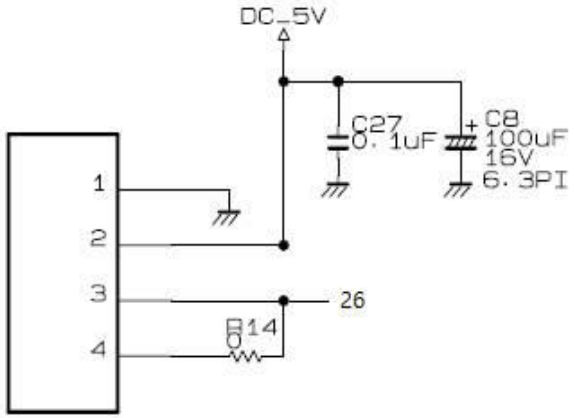
```c
while(1)
    {
        if(duty > 90) updown = -10;
        if(duty < 20) updown = 10;
        duty += updown;
        printf("Motor duty = %d\n", duty);

        softPwmWrite(DCMotorA, duty);
        softPwmWrite(DCMotorB, 0);
        printf("Motor run CCW\n");
        delay(3000);

        softPwmWrite(DCMotorA, 0);
        softPwmWrite(DCMotorB, 0);
        printf("Motor STOP\n");
        delay(1000);

        softPwmWrite(DCMotorA, 0);
        softPwmWrite(DCMotorB, duty);
        printf("Motor run CW\n");
        delay(3000);

        softPwmWrite(DCMotorA, 0);
        softPwmWrite(DCMotorB, 0);
        printf("Motor STOP\n");
        delay(1000);
    }
}
```

Servo Motor(1/3)

Servo motor

- PWM을 사용하여 회전각을 조절할 수 있는 모터
- 내부의 기계적인 정밀도에 따라 좀 더 정밀한 제어가 가능하다.

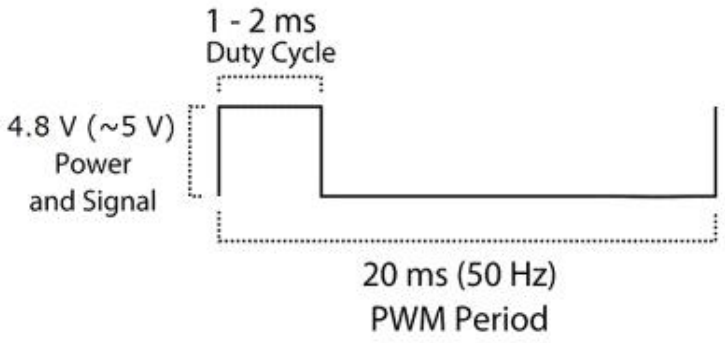


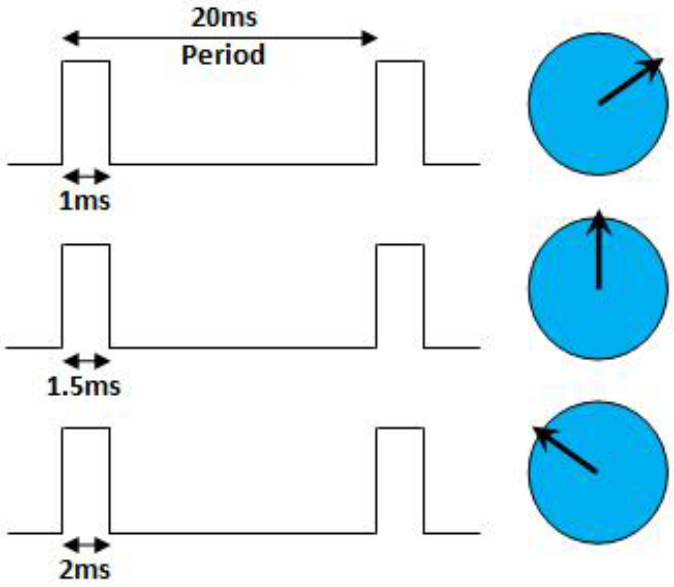


Servo Motor(2/3)

Servo motor의 동작 원리

- 20ms 주기로 1~2ms 펄스폭의 PWM 파형을 입력 받아서 동작





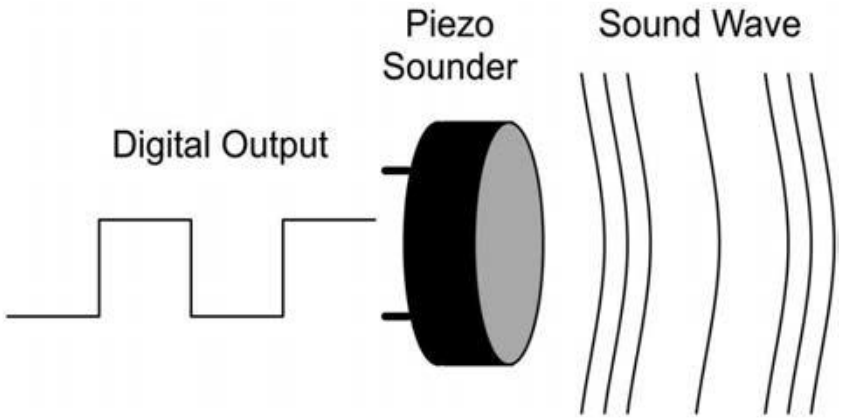
Servo Motor(3/3)

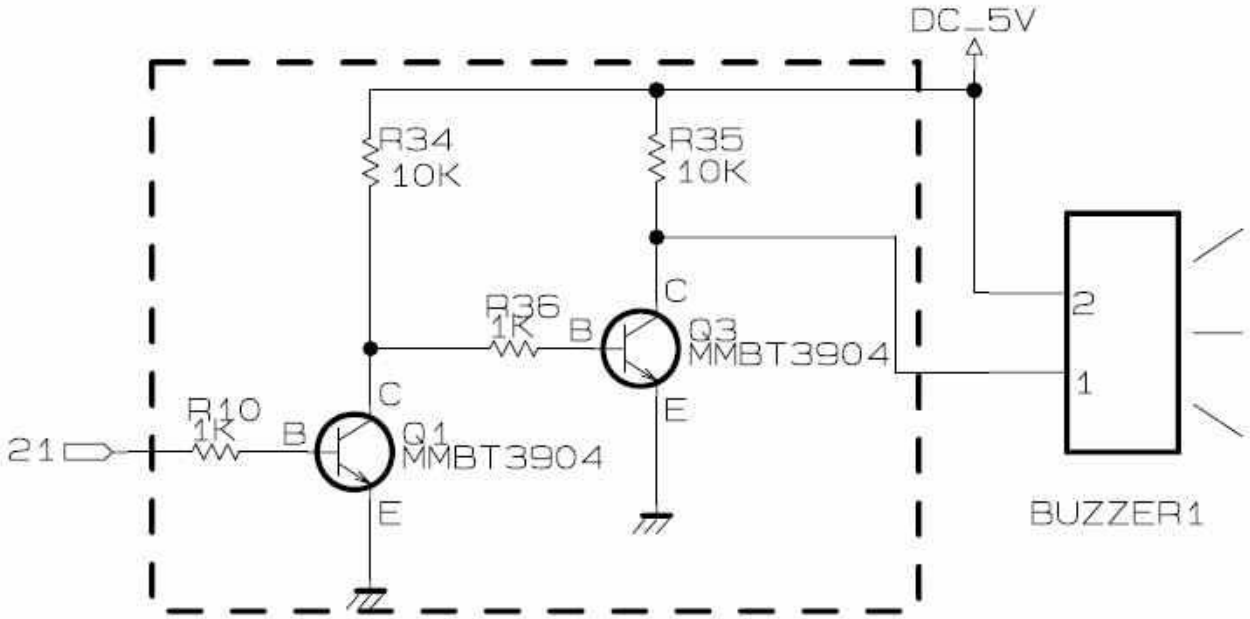
```c
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>
#define SERVO_PIN 26

int main(void){

    if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }

    pinMode(SERVO_PIN, SOFT_PWM_OUTPUT);
    softPwmCreate(SERVO_PIN, 0, 200 );
    while(1){
        //- 15도 방향 이동
        softPwmWrite(SERVO_PIN, 15);
        delay(500);
        //- 24도 방향 이동
        softPwmWrite(SERVO_PIN, 24);
        delay(500);
        //- 5도 방향 이동
        softPwmWrite(SERVO_PIN, 5);
        delay(500);
    }
    softPwmStop(SERVO_PIN);
    delay(500);
    return 0;
}
```

Passive Buzzer(1/3)

Passive Buzzer

- 출력 주파수의 수를 조절하여 다양한 높낮이의 소리를 출력할 수 있는 부저

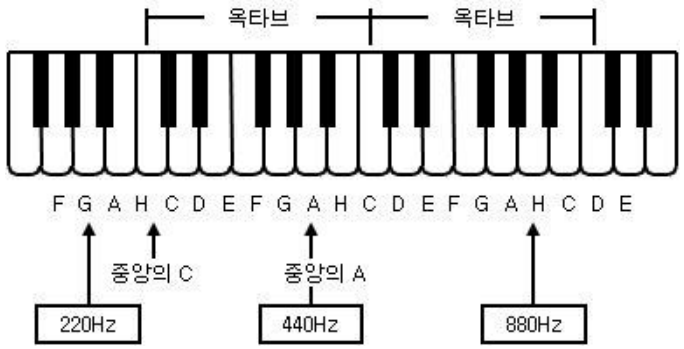




Passive Buzzer(2/3)

주파수를 이용한 제어

- 음역대에 해당하는 주파수 만큼 on/off 반복하여 음의 높낮이를 연주



(단위 : Hz)

옥타브 / 음계	1	2	3	4	5	6	7	8
C(도)	32.7032	65.4064	130.8128	261.6256	523.2511	1046.502	2093.005	4186.009
C#	34.6478	69.2957	138.5913	277.1826	554.3653	1108.731	2217.461	4434.922
D(레)	36.7081	73.4162	146.8324	293.6648	587.3295	1174.659	2349.318	4698.636
D#	38.8909	77.7817	155.5635	311.1270	622.2540	1244.508	2489.016	4978.032
E(미)	41.2034	82.4069	164.8138	329.6276	659.2551	1318.510	2637.020	5274.041
F(파)	43.6535	87.3071	174.6141	349.2282	698.4565	1396.913	2793.826	5587.652
F#	46.2493	92.4986	184.9972	369.9944	739.9888	1479.978	2959.955	5919.911
G(솔)	48.9994	97.9989	195.9977	391.9954	783.9909	1567.982	3135.963	6271.927
G#	51.9130	103.8262	207.6523	415.3047	830.6094	1661.219	3322.438	6644.875
A(라)	55.0000	110.0000	220.0000	440.0000	880.0000	1760.000	3520.000	7040.000
A#	58.2705	116.5409	233.0819	466.1638	932.3275	1864.655	3729.310	7458.620
B(시)	61.7354	123.4708	246.9417	493.8833	987.7666	1975.533	3951.066	7902.133

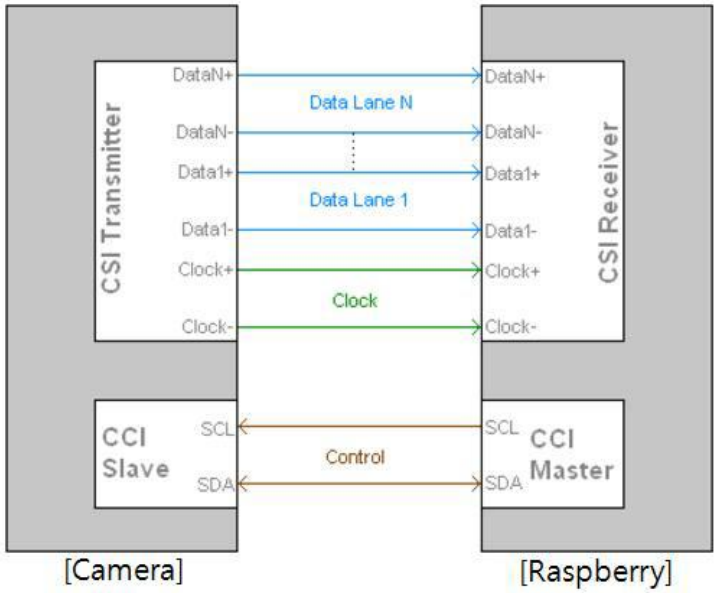
Passive Buzzer(3/3)

```
#include <wiringPi.h>
#include <softTone.h>
#define BUZ_P_PIN   21     //- WPI

unsigned int melody[8] = {262, 294, 330, 349, 392, 440, 494, 523};

int main(void)
{
    int idx = 0;

    if( wiringPiSetup() == -1)  exit(1);    //- wiringPi 초기화

    pinMode(BUZ_P_PIN, SOFT_TONE_OUTPUT);

    softToneCreate(BUZ_P_PIN);               //- 주파수 출력 설정

    while(1){
        for(idx=0; idx<8; idx++){
          softToneWrite(BUZ_P_PIN, melody[idx]);
          delay(1000);
        }
    }
    softToneStop(BUZ_P_PIN);    //- 주파수 출력 중단
}
```

Camera(1/6)

Pi Camera 모듈

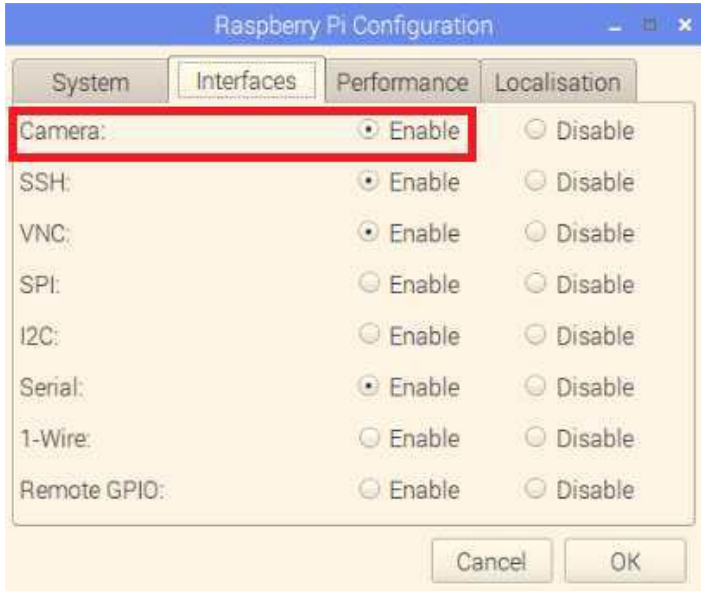
- 카메라 모듈은 리본 케이블을 사용해 연결
- I2C 방식을 통해서 데이터 송수신



카메라 모듈의 통신 구조도

Camera(2/6)

Pi Camera 시스템 설정



시스템의 설정 과정

Camera(3/6)

Libcamera shell 명령어

- libcamera-hello : 미리보기 창으로 카메라에 보여지는 화면을 확인
- 원격접속으로는 지원되지 않는다.

```
pi@raspberrypi:~ $ libcamera-hello --help
Valid options are:
  -h [ --help ] [=arg(=1)] (=0)         Print this help message
  --version [=arg(=1)] (=0)             Displays the build version number
  --list-cameras [=arg(=1)] (=0)        Lists the available cameras attached to the system.
  --camera arg (=0)                     Chooses the camera to use. To list the available indexes, use the
                                        --list-cameras option.
  -v [ --verbose ] [=arg(=2)] (=1)      Set verbosity level. Level 0 is no output, 1 is default, 2 is verbose.
  -c [ --config ] [=arg(=config.txt)]   Read the options from a file. If no filename is specified, default to
                                        config.txt. In case of duplicate options, the ones provided on the command line
                                        will be used. Note that the config file must only contain the long form
                                        options.
  --info-text arg (=#%frame (%fps fps) exp %exp ag %ag dg %dg)
                                        Sets the information string on the titlebar. Available values:
                                        %frame (frame number)
                                        %fps (framerate)
                                        %exp (shutter speed)
                                        %ag (analogue gain)
                                        %dg (digital gain)
                                        %rg (red colour gain)
                                        %bg (blue colour gain)
                                        %focus (focus FoM value)
                                        %aelock (AE locked status)
                                        %lp (lens position, if known)
                                        %afstate (AF state, if supported)
  --width arg (=0)                      Set the output image width (0 = use default value)
  --height arg (=0)                     Set the output image height (0 = use default value)
  -t [ --timeout ] arg (=5000)          Time (in ms) for which program runs
  -o [ --output ] arg                   Set the output file name
  --post-process-file arg               Set the file name for configuring the post-processing
  --rawfull [=arg(=1)] (=0)             Force use of full resolution raw frames
  -n [ --nopreview ] [=arg(=1)] (=0)    Do not show a preview window
  -p [ --preview ] arg (=0,0,0,0)       Set the preview window dimensions, given as x,y,width,height e.g. 0,0,640,480
  -f [ --fullscreen ] [=arg(=1)] (=0)   Use a fullscreen preview window
  --qt-preview [=arg(=1)] (=0)          Use Qt-based preview window (WARNING: causes heavy CPU load, fullscreen not
                                        supported)
```

Camera(4/6)

Libcamera shell 명령어

- libcamera-jpeg : 사진을 촬영하는 명령어
- 원격에서는 미리보기 창은 지원되지 않으나 저장된 파일은 확인 가능하다.

```
pi@raspberrypi:~ $ libcamera-jpeg --help
Valid options are:
  -h [ --help ] [=arg(=1)] (=0)         Print this help message
  --version [=arg(=1)] (=0)             Displays the build version number
  --list-cameras [=arg(=1)] (=0)        Lists the available cameras attached to the system.
  --camera arg (=0)                     Chooses the camera to use. To list the available indexes, use the
                                        --list-cameras option.
  -v [ --verbose ] [=arg(=2)] (=1)      Set verbosity level. Level 0 is no output, 1 is default, 2 is verbose.
  -c [ --config ] [=arg(=config.txt)]   Read the options from a file. If no filename is specified, default to
                                        config.txt. In case of duplicate options, the ones provided on the command line
                                        will be used. Note that the config file must only contain the long form
                                        options.
  --info-text arg (=#%frame (%fps fps) exp %exp ag %ag dg %dg)
                                        Sets the information string on the titlebar. Available values:
                                        %frame (frame number)
                                        %fps (framerate)
                                        %exp (shutter speed)
                                        %ag (analogue gain)
                                        %dg (digital gain)
                                        %rg (red colour gain)
                                        %bg (blue colour gain)
                                        %focus (focus FoM value)
                                        %aelock (AE locked status)
                                        %lp (lens position, if known)
                                        %afstate (AF state, if supported)
  --width arg (=0)                      Set the output image width (0 = use default value)
  --height arg (=0)                     Set the output image height (0 = use default value)
  -t [ --timeout ] arg (=5000)          Time (in ms) for which program runs
  -o [ --output ] arg                   Set the output file name
  --post-process-file arg               Set the file name for configuring the post-processing
  --rawfull [=arg(=1)] (=0)             Force use of full resolution raw frames
  -n [ --nopreview ] [=arg(=1)] (=0)    Do not show a preview window
  -p [ --preview ] arg (=0,0,0,0)       Set the preview window dimensions, given as x,y,width,height e.g. 0,0,640,480
  -f [ --fullscreen ] [=arg(=1)] (=0)   Use a fullscreen preview window
  --qt-preview [=arg(=1)] (=0)          Use Qt-based preview window (WARNING: causes heavy CPU load, fullscreen not
                                        supported)
```

Camera(5/6)

Libcamera shell 명령어

- libcamera-still : 사진을 촬영하는 명령어. 더 많은 옵션을 가지고 있다.
  - 다양한 이미지 포멧으로 저장 가능
- libcamera-raw : 카메라에서 인식한 원본 이미지 데이터를 저장
- libcamera-vid : 비디오를 촬영하는 명령어
  - H.264 포멧을 기본으로 사용
  - ex) libcamera-vid -t 20000 -o vid_test.h264 -> 20초 동안 촬영된 비디오를 vid_test.h264 파일로 저장

Camera(6/6)

Libcamera 기본 예제

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{
    while(1)
    {

        getchar();

        char cmd[] = "sudo libcamera-jpeg --width 800 --height 800 -t 1 -o test.jpeg";
        printf("%s \n", cmd);
        system(cmd);
        printf("Take a picture \n");
    }

    return 0;
}
```

getch() 함수

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <termios.h>
char getch();
int main(void){
char key;
if(key=getch()){
      switch(key){
        case 'q':
          break;
        case 'e':
          break;
      }
}
```

```
char getch(){
   char c;
   struct termios oldattr, newattr;

   tcgetattr(STDIN_FILENO, &oldattr);      // now terminal config
   newattr = oldattr;
   newattr.c_lflag &= ~(ICANON | ECHO);  // turn off CANONICAL and ECHO
   newattr.c_cc[VMIN] = 1;                                  // least letter count 1
   newattr.c_cc[VTIME] = 0;                                 // least read wating time 0
   tcsetattr(STDIN_FILENO, TCSANOW, &newattr);      // input terminal config
   c = getchar();                                                          // read keyboard
   tcsetattr(STDIN_FILENO, TCSANOW, &oldattr);      // recover original coonfig
   return c;
}
```

과제 6

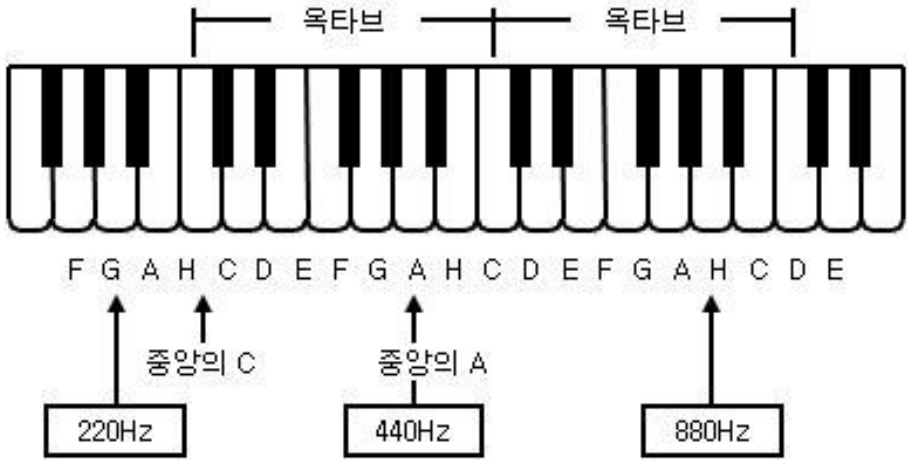
키보드 입력을 통해 서보 모터를 제어하는 프로그램을 만드시오.

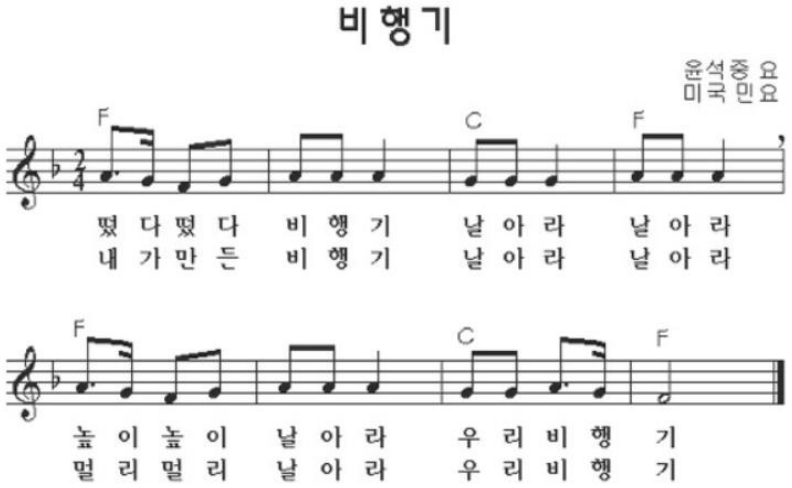
- 서보모터는 초기에 오른쪽을 가리키고 있다. (15 에 해당하는 각도)
- 키보드의 q 를 누르면 서보 모터가 반시계 방향 1만큼 회전한다. (최대 24)
- 키보드의 e 를 누르면 서보 모터가 시계 방향 1만큼 회전한다. (최소 6)
- 이 때 각 버튼은 enter를 누르지 않고, 누르는 즉시 이동한다.
- 함수는 다음 장의 char getch() 함수를 사용한다.
- 이와 동시에, 서보 모터의 각도가 가장 작을 때는 LED가 꺼져 있고 상승할 수록 단계적으로 밝아진다.

과제 7

수동 부저를 사용하여 피아노 프로그램을 만드시오.

- a~k 의 키보드 버튼은 각각 ‘낮은 도’ ~ ‘높은 도’에 대응한다.
- getch() 함수를 사용하여 8개의 음을 연주하는 피아노를 만드시오.
- 키 한 번은 125 ms 의 딜레이를 갖는다.





과제 8

카메라를 이용하여 접근하는 물체를 촬영하시오.

- 초기에는 초음파 센서, 카메라, 부저가 동작하지 않는다.
- enter 키를 누르면 시스템이 시작된다. 시스템이 시작되면 초음파 센서를 통해 거리를 측정한다.
- 만약 물체가 20cm 이내로 접근할 경우, 부저를 3초 동안 짧은 주기로 울린다.
- 만약 부저가 울린 이후에도 물체가 20cm 이내에 있을 경우 사진을 촬영한다.
- 사진을 촬영하게 되면 “Take a Picture!” 문장을 출력한다.
- 부저가 울린 이후에 물체가 20cm에 있지 않다면 아무 일도 일어나지 않는다.

고생하셨습니다

Thank you!